

EDA Basket Analysis Beliz & Elizaveta

2026-04-17

Exploratory Data Analysis

Libraries

The analysis uses the tidyverse for data manipulation and visualization, knitr and kableExtra for table formatting in the report, and hexbin for density-based visualizations (hexagonal binning).

```
library(tidyverse)
library(knitr)
library(kableExtra)
library(hexbin)
```

Raw Dataset

The dataset is read as raw text using readLines(), where each line represents a single transaction. A tibble is then created with a unique transaction_id for each row and a basket column containing the raw transaction string. The head() function is used to inspect the first observations.

```
lines <- readLines("~/Desktop/supermarket case/supermarket.csv", warn = FALSE)

supermarket_raw <- tibble(
  transaction_id = 1:length(lines),
  basket = lines
)

head(supermarket_raw)

## # A tibble: 6 x 2
##   transaction_id basket
##           <int> <chr>
## 1             1 "10 22 3.18, 10 173 6.56, 18 93 4.31, 12 46 6.09, 12 51 3.94, ~
## 2             2 "10 45 2.72, 18 94 4.52, 7 137 8.91, 13 16 0.34, 6 317 0.75, 1~
## 3             3 "10 43 6.35, 10 63 4.02, 10 45 6.84, 15 95 1.79, 15 78 2.08, 1~
## 4             4 "10 26 1.17, 14 83 0.72, 18 61 1.03, 18 51 1.55, 1 353 2.59, 1~
## 5             5 "10 46 0.9, 10 36 1.61, 18 242 0.89, 18 62 2.34, 18 52 0.86, 1~
## 6             6 "10 53 3.01, 10 39 10.59, 10 58 0.24, 10 58 5.56, 10 43 3.39, ~
```

Transformation into a structured format

The raw transaction strings are transformed into a structured format. Each basket is split into individual product entries, and each entry is further separated into department, time (between scans), and price.

These variables are then converted to numeric types. Finally, the data is inspected using `head()`, `structure`, summary statistics, and a check of unique transaction IDs.

```
long_data <- supermarket_raw %>%
  separate_rows(basket, sep = ",") %>%
  mutate(basket = str_trim(basket)) %>%
  separate(
    basket,
    into = c("department", "time", "price"),
    sep = "\\s+",
    fill = "right",
    extra = "drop"
  ) %>%
  mutate(
    department = as.integer(department),
    time = as.numeric(time),
    price = as.numeric(price)
  )

head(long_data)
```

```
## # A tibble: 6 x 4
##   transaction_id department   time price
##           <int>       <int> <dbl> <dbl>
## 1             1         10    22  3.18
## 2             1         10   173  6.56
## 3             1         18    93  4.31
## 4             1         12    46  6.09
## 5             1         12    51  3.94
## 6             1         13   285  1.71
```

```
str(long_data)
```

```
## tibble [2,129,941 x 4] (S3: tbl_df/tbl/data.frame)
## $ transaction_id: int [1:2129941] 1 1 1 1 1 1 1 1 1 1 ...
## $ department   : int [1:2129941] 10 10 18 12 12 13 15 15 15 15 ...
## $ time         : num [1:2129941] 22 173 93 46 51 285 78 54 173 40 ...
## $ price        : num [1:2129941] 3.18 6.56 4.31 6.09 3.94 1.71 2.84 1.02 0.1 0.62 ...
```

```
summary(long_data)
```

```
## transaction_id      department      time      price
## Min.   :      1   Min.   : 1.00   Min.   :  0.0   Min.   : 0.00
## 1st Qu.: 25084   1st Qu.: 6.00   1st Qu.: 37.0   1st Qu.: 1.15
## Median : 50080   Median :10.00   Median : 60.0   Median : 2.00
## Mean   : 50063   Mean   :10.14   Mean   :100.1   Mean   : 2.86
## 3rd Qu.: 75037   3rd Qu.:15.00   3rd Qu.:116.0   3rd Qu.: 3.88
## Max.   :100000   Max.   :18.00   Max.   :2420.0   Max.   :29.63
##          NA's    :1e+05   NA's    :1e+05   NA's    :1e+05
```

```
length(unique(long_data$transaction_id))
```

```
## [1] 100000
```

The dataset contains over 2.1 million item-level observations across 100,000 transactions.

The distribution of time between scans is right-skewed, with a median of 60 seconds but extreme values reaching up to 2,420 seconds, indicating the presence of long pauses.

Price distribution is also right-skewed, with most items priced around €2, but occasional high-value purchases.

Additionally, approximately 5% of the data contains missing values, which requires further data cleaning.

Mapping

A mapping is created to translate numerical department codes into descriptive category names, then applied to the dataset, adding a new variable (`department_name`) that improves interpretability. The updated data is inspected using `head()`.

```
dept_map <- c(
  "1" = "Bakery & Pastry",
  "2" = "Beer & Wine",
  "3" = "Books & Magazines",
  "4" = "Candy & Chips",
  "5" = "Care & Hygiene",
  "6" = "Cereals & Spreads",
  "7" = "Cheese & Tapas",
  "8" = "Dairy & Eggs",
  "9" = "Freezer",
  "10" = "Fruit & Vegetables",
  "11" = "Household & Pet",
  "12" = "Meat & Fish",
  "13" = "Pasta & Rice",
  "14" = "Salads & Meals",
  "15" = "Sauces & Spices",
  "16" = "Soda & Juices",
  "17" = "Special Diet",
  "18" = "Vegetarian & Vegan"
)

long_data <- long_data %>%
  mutate(department_name = dept_map[as.character(department)])

head(long_data)
```

```
## # A tibble: 6 x 5
##   transaction_id department   time price department_name
##           <int>      <int> <dbl> <dbl> <chr>
## 1             1         10    22   3.18 Fruit & Vegetables
## 2             1         10   173   6.56 Fruit & Vegetables
## 3             1         18    93   4.31 Vegetarian & Vegan
## 4             1         12    46   6.09 Meat & Fish
```

```
## 5          1          12    51  3.94 Meat & Fish
## 6          1          13   285  1.71 Pasta & Rice
```

Numeric department codes were mapped to descriptive category names. This transformation allows for a more meaningful analysis of customer purchasing behavior, enabling comparisons across product categories and identification of common shopping patterns.

Data Cleaning

The dataset is cleaned by removing observations with missing or invalid values (for example, NA, negative time or price). A new variable, `price_per_second`, is then created to capture the ratio of price to time between scans, serving as an additional behavioral feature.

```
long_data_clean <- long_data %>%
  filter(
    !is.na(department),
    !is.na(time),
    !is.na(price),
    time >= 0,
    price >= 0
  ) %>%
  mutate(
    price_per_second = ifelse(time > 0, price / time, NA_real_)
  )
```

Descriptive Analysis of Basket Characteristics

The data is aggregated at the transaction level, turning item-level data into basket-level features. For each transaction, key metrics are calculated, including basket size (`total_items`), total and average price, total and average time between scans, variability measures, and price-related indicators. The number of unique departments is also included.

The resulting dataset is then checked using basic outputs such as `head()`, `structure`, and summary statistics.

```
basket_data <- long_data_clean %>%
  group_by(transaction_id) %>%
  summarise(
    total_items = n(),
    total_price = sum(price, na.rm = TRUE),
    avg_price = mean(price, na.rm = TRUE),
    total_time = sum(time, na.rm = TRUE),
    avg_time_per_item = mean(time, na.rm = TRUE),
    sd_time_per_item = sd(time, na.rm = TRUE),
    max_time = max(time, na.rm = TRUE),
    min_time = min(time, na.rm = TRUE),
    price_per_second = ifelse(total_time > 0, total_price / total_time, NA_real_),
    max_price = max(price, na.rm = TRUE),
    min_price = min(price, na.rm = TRUE),
    unique_departments = n_distinct(department),
    .groups = "drop"
  )

head(basket_data)
```

```
## # A tibble: 6 x 13
##   transaction_id total_items total_price avg_price total_time avg_time_per_item
##           <int>       <int>       <dbl>     <dbl>       <dbl>         <dbl>
## 1             1         31        77.4       2.50        2361          76.2
## 2             2         23        67.1       2.92        1823          79.3
## 3             3         24        83.8       3.49        2551         106.
## 4             4         33        77.1       2.34        3114          94.4
## 5             5         19        28.6       1.51        1953         103.
## 6             6         14        58.0       4.15        1798         128.
## # i 7 more variables: sd_time_per_item <dbl>, max_time <dbl>, min_time <dbl>,
## #   price_per_second <dbl>, max_price <dbl>, min_price <dbl>,
## #   unique_departments <int>
```

```
str(basket_data)
```

```
## tibble [100,000 x 13] (S3: tbl_df/tbl/data.frame)
## $ transaction_id : int [1:100000] 1 2 3 4 5 6 7 8 9 10 ...
## $ total_items    : int [1:100000] 31 23 24 33 19 14 15 36 12 19 ...
## $ total_price    : num [1:100000] 77.4 67.1 83.8 77.1 28.6 ...
## $ avg_price      : num [1:100000] 2.5 2.92 3.49 2.34 1.51 ...
## $ total_time     : num [1:100000] 2361 1823 2551 3114 1953 ...
## $ avg_time_per_item : num [1:100000] 76.2 79.3 106.3 94.4 102.8 ...
## $ sd_time_per_item : num [1:100000] 60.1 84.7 68.2 79.3 115.5 ...
## $ max_time       : num [1:100000] 285 317 280 353 464 711 426 557 735 287 ...
## $ min_time       : num [1:100000] 22 11 11 15 19 27 25 18 19 25 ...
## $ price_per_second : num [1:100000] 0.0328 0.0368 0.0329 0.0248 0.0147 ...
## $ max_price       : num [1:100000] 6.56 8.91 8.5 8.96 4.65 ...
## $ min_price       : num [1:100000] 0.1 0.34 0.71 0.21 0.44 0.24 0.3 0.08 0.08 0.19 ...
## $ unique_departments: int [1:100000] 7 12 9 14 9 7 5 12 8 12 ...
```

```
summary(basket_data)
```

```
## transaction_id    total_items    total_price    avg_price
## Min.   :      1    Min.   :  3.0    Min.   :  0.41    Min.   : 0.1367
## 1st Qu.: 25001    1st Qu.: 13.0    1st Qu.: 31.68    1st Qu.: 1.9236
## Median : 50000    Median : 19.0    Median : 48.56    Median : 2.4653
## Mean   : 50000    Mean   : 20.3    Mean   : 58.06    Mean   : 2.8654
## 3rd Qu.: 75000    3rd Qu.: 25.0    3rd Qu.: 74.92    3rd Qu.: 3.7245
## Max.   :100000    Max.   :108.0    Max.   :409.14    Max.   :11.0350
## total_time      avg_time_per_item sd_time_per_item    max_time
## Min.   :    59    Min.   : 19.67    Min.   :  5.627    Min.   :   33.0
## 1st Qu.: 1362    1st Qu.: 81.13    1st Qu.: 69.407    1st Qu.: 274.0
## Median : 1867    Median : 99.28    Median : 94.848    Median : 376.0
## Mean   : 2032    Mean   :106.62    Mean   :107.390    Mean   : 423.6
## 3rd Qu.: 2507    3rd Qu.:123.08    3rd Qu.:130.931    3rd Qu.: 520.0
## Max.   :11027    Max.   :830.67    Max.   :947.009    Max.   :2420.0
## min_time        price_per_second    max_price    min_price
## Min.   :  0.00    Min.   :0.0006766    Min.   :  0.190    Min.   :0.000
## 1st Qu.: 15.00    1st Qu.:0.0183082    1st Qu.:  5.150    1st Qu.:0.130
## Median : 20.00    Median :0.0256522    Median :  7.030    Median :0.310
## Mean   : 21.63    Mean   :0.0297588    Mean   :  7.964    Mean   :0.405
## 3rd Qu.: 26.00    3rd Qu.:0.0378401    3rd Qu.:  9.900    3rd Qu.:0.560
## Max.   :569.00    Max.   :0.1691908    Max.   :29.630    Max.   :7.450
```

```
## unique_departments
## Min.      : 1.00
## 1st Qu.: 7.00
## Median : 9.00
## Mean    : 9.06
## 3rd Qu.:11.00
## Max.     :18.00
```

Summary Statistic Table

A summary statistics table is created for the main basket-level variables. The selected variables are reshaped into long format, grouped by metric, and summarized using mean, median, standard deviation, selected percentiles, and maximum values. The metric names are then relabeled for readability, numeric values are rounded, and the final table is formatted using kable and kableExtra for clear presentation in the report.

```
stats_table <- basket_data %>%
  select(total_items, total_price, total_time, avg_time_per_item, sd_time_per_item) %>%
  pivot_longer(cols = everything(), names_to = "Metric", values_to = "Value") %>%
  group_by(Metric) %>%
  summarise(
    Mean = mean(Value, na.rm = TRUE),
    Median = median(Value, na.rm = TRUE),
    SD = sd(Value, na.rm = TRUE),
    P25 = quantile(Value, 0.25, na.rm = TRUE),
    P75 = quantile(Value, 0.75, na.rm = TRUE),
    Max = max(Value, na.rm = TRUE)
  )

stats_table <- stats_table %>%
  mutate(Metric = recode(Metric,
    total_items = "Number of items",
    total_price = "Total price (€)",
    total_time = "Total scan time (sec)",
    avg_time_per_item = "Avg time per item (sec)",
    sd_time_per_item = "Scan time variability"
  ),
  across(-Metric, ~round(.x, 2))
  )

kable(stats_table, digits = 2, align = "lrrrrrr",
  caption = "Summary statistics of customer basket behavior", booktabs = TRUE) %>%
  kable_styling(
    latex_options = c("hold_position", "striped"),
    full_width = FALSE,
    position = "center"
  ) %>%
  column_spec(1, bold = TRUE) %>%
  add_header_above(c(" " = 1, "Statistics" = 6))
```

Table 1: Summary statistics of customer basket behavior

	Statistics					
--	------------	--	--	--	--	--

Metric	Mean	Median	SD	P25	P75	Max
Avg time per item (sec)	106.62	99.28	38.90	81.13	123.08	830.67
Scan time variability	107.39	94.85	56.47	69.41	130.93	947.01
Number of items	20.30	19.00	9.81	13.00	25.00	108.00
Total price (€)	58.06	48.56	37.55	31.68	74.92	409.14
Total scan time (sec)	2032.48	1867.00	963.94	1362.00	2507.00	11027.00

The typical customer basket contains around 19-20 items, with total spending around €48.6. The distribution of total price is right-skewed, meaning there are some higher-value transactions.

Customers spend about 30 minutes per shopping session, with most trips lasting between 23 and 42 minutes.

The average time per item is relatively high (around 99 seconds), and there is quite a lot of variation within baskets. This suggests that customers do not scan items continuously, but take pauses as part of the shopping process.

Finally, the presence of extreme values in time-related variables points to outliers, such as very long pauses or possibly unclosed sessions.

Basket Analysis

Basket Size

The distribution of basket size is right-skewed, with most customers buying a moderate number of items and a long tail of larger baskets. The median basket contains 19 items, while the mean is slightly higher at 20.3, pulled up by those larger shopping trips. The middle 50% of customers buy between 13 and 25 items (IQR), which can be considered the “normal” basket size range. The boxplot confirms this, with outliers stretching up to 108 items, which are likely bulk purchases or large household shopping trips. For the typical customer, a normal basket contains roughly 13-25 items, with 19 being the most representative single value.

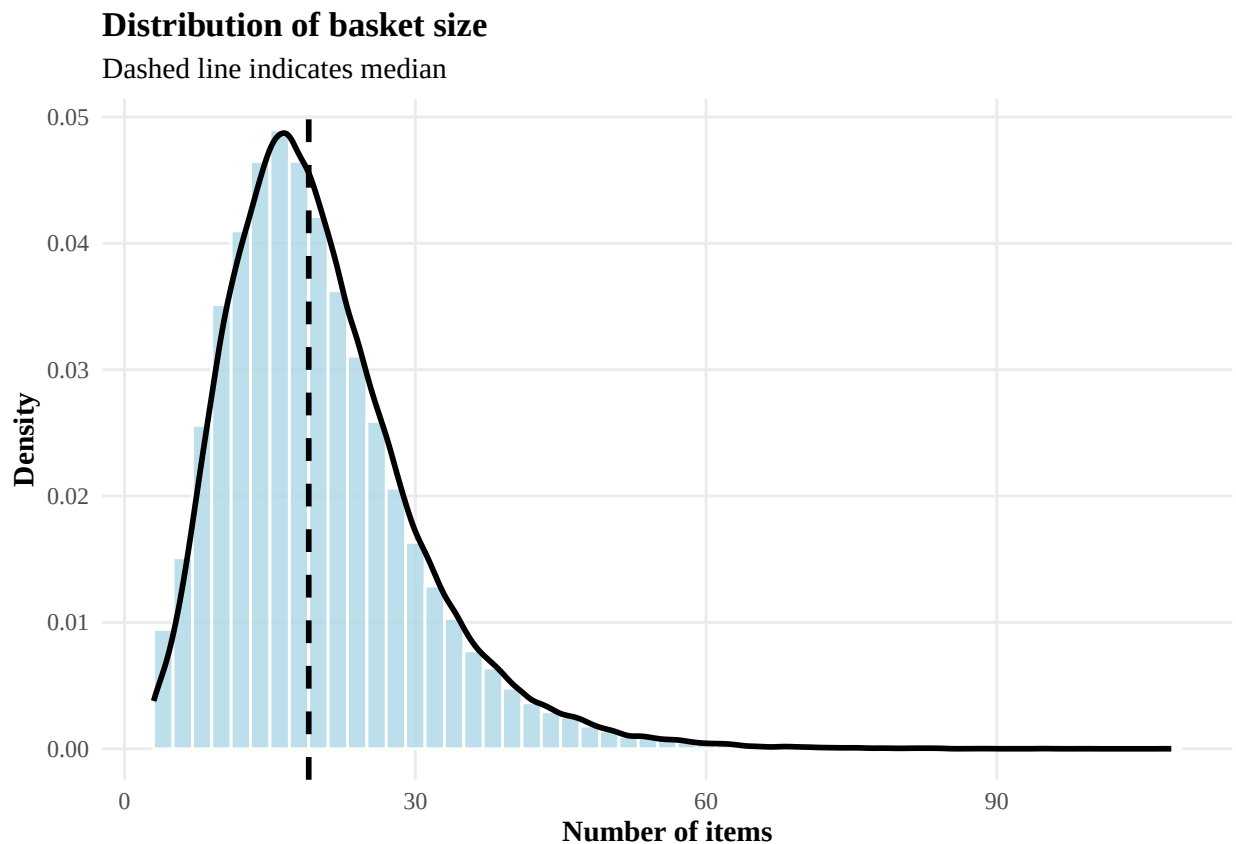
Total Basket Value (Spending Range)

Spending follows a similar right-skewed pattern. The median spend is €48.56, meaning half of all customers spend less than €50 per trip. The interquartile range runs from €31.68 to €74.92, capturing the core spending behaviour of most customers. There are notable high-value outliers reaching up to €409, but these are rare. The vast majority of baskets fall well below €150 as seen in both the following histogram and boxplot. A typical customer spends between €32 and €75, with €49 being a strong central estimate.

Distribution of Basket Size

```
ggplot(basket_data, aes(x = total_items)) +
  geom_histogram(aes(y = after_stat(density)),
    binwidth = 2,
    fill = "lightblue",
    color = "white",
    alpha = 0.8) +
  geom_density(color = "black", linewidth = 1) +
  geom_vline(aes(xintercept = median(total_items)),
    linetype = "dashed",
    linewidth = 1) +
```

```
labs(
  title = "Distribution of basket size",
  subtitle = "Dashed line indicates median",
  x = "Number of items",
  y = "Density"
) +
theme_minimal(base_family = "serif") +
theme(
  plot.title = element_text(face = "bold"),
  axis.title = element_text(face = "bold"),
  panel.grid.minor = element_blank()
)
```



The distribution peaks around 15-20 items and then gradually tapers off to the right. It is clearly right-skewed and the bulk of customers cluster in the 10-30 item range, but a long tail extends toward 60-90+ items. This tells you that the “typical” shopper buys a moderate amount, while very large baskets are uncommon but do exist.

Distribution of Total Basket Value

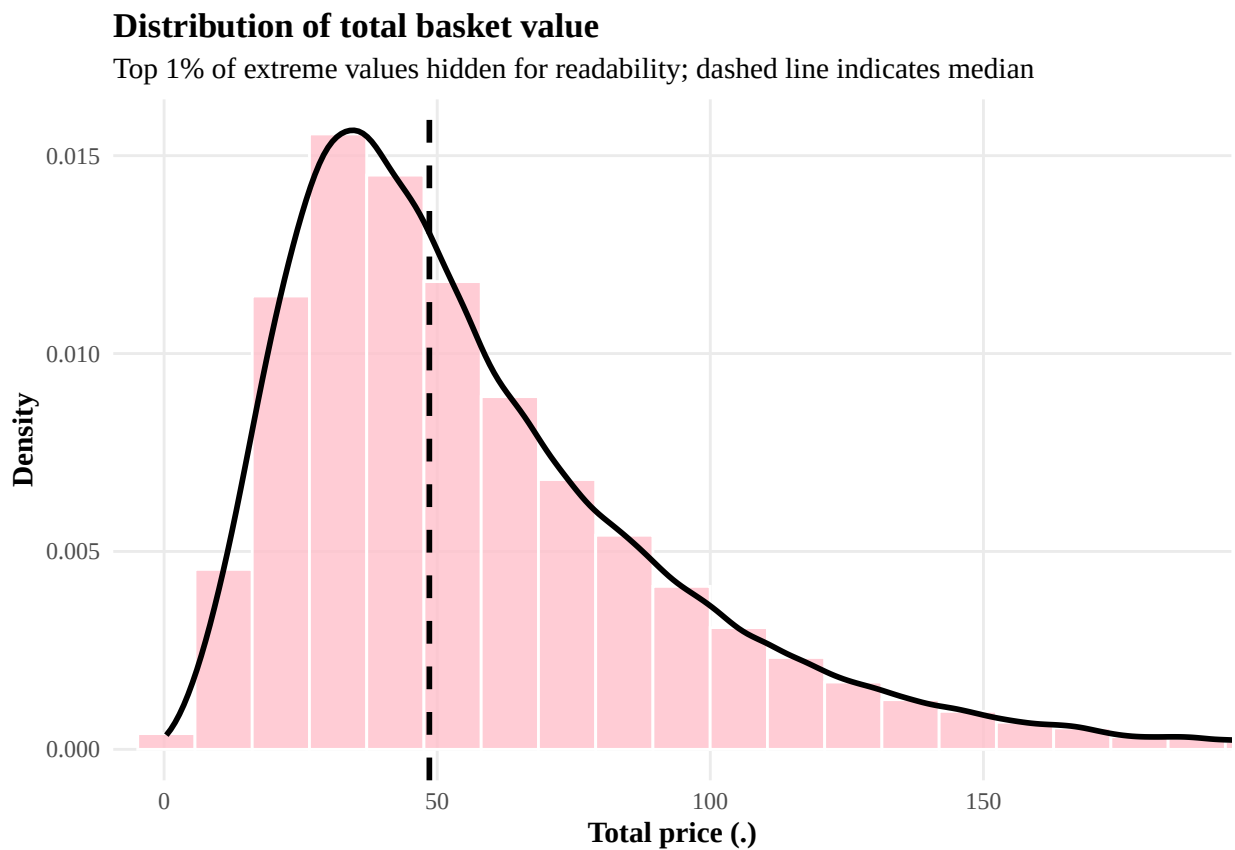
```
ggplot(basket_data, aes(x = total_price)) +
  geom_histogram(aes(y = after_stat(density)),
    bins = 40,
    fill = "pink",
```



```

        color = "white",
        alpha = 0.8) +
geom_density(color = "black", linewidth = 1) +
geom_vline(aes(xintercept = median(total_price, na.rm = TRUE)),
           linetype = "dashed",
           linewidth = 1) +
coord_cartesian(
  xlim = c(0, quantile(basket_data$total_price, 0.99, na.rm = TRUE))
) +
labs(
  title = "Distribution of total basket value",
  subtitle = "Top 1% of extreme values hidden for readability; dashed line indicates median",
  x = "Total price (€)",
  y = "Density"
) +
theme_minimal(base_family = "serif") +
theme(
  plot.title = element_text(face = "bold"),
  axis.title = element_text(face = "bold"),
  panel.grid.minor = element_blank()
)

```



```

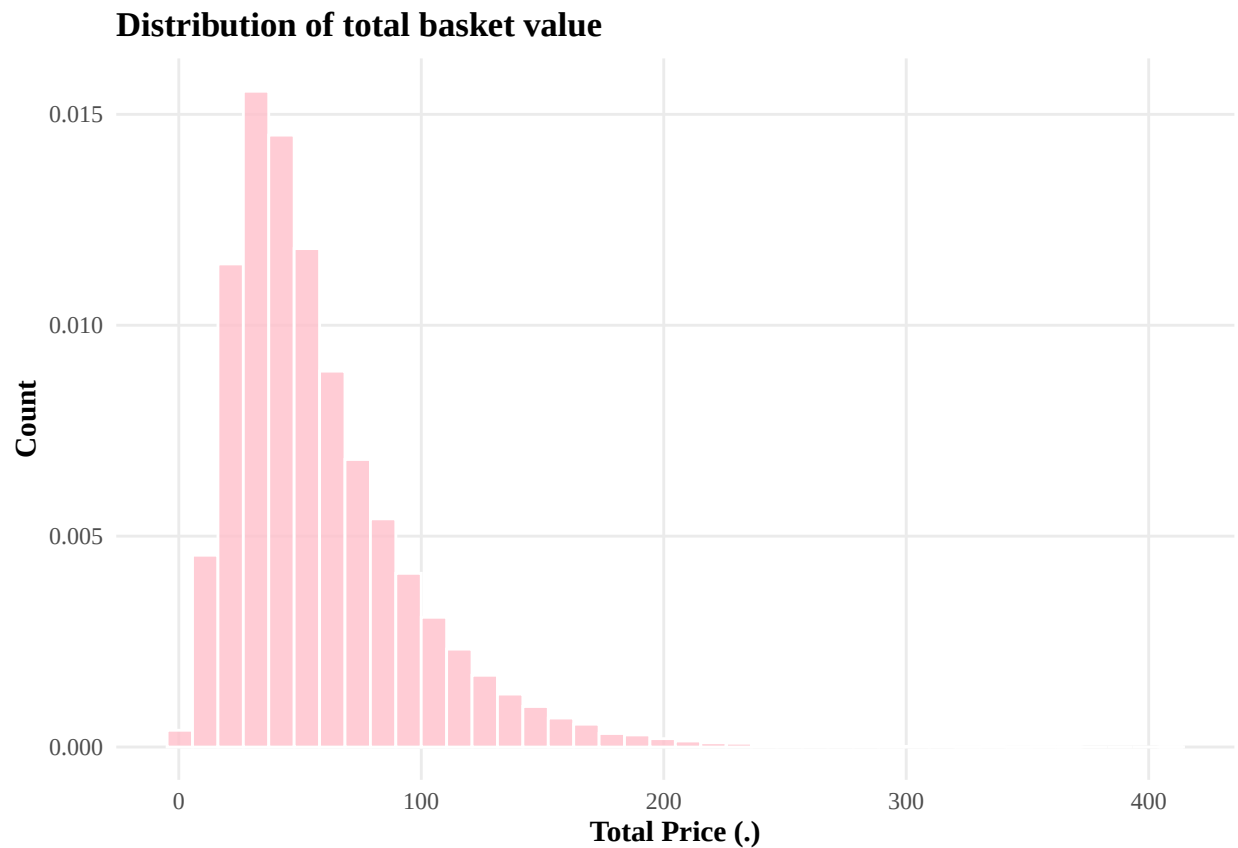
ggplot(basket_data, aes(x = total_price)) +
  geom_histogram(aes(y = after_stat(density)),
                bins = 40,

```

```

    fill = "pink",
    color = "white",
    alpha = 0.8) +
labs(title = "Distribution of total basket value", x = "Total Price (€)", y = "Count") +
theme_minimal(base_family = "serif") +
theme(
  plot.title = element_text(face = "bold"),
  axis.title = element_text(face = "bold"),
  panel.grid.minor = element_blank()
)

```



Spending is also right-skewed, peaking around €20–€35 and taking off steeply after €100. The long tail reaching €400 represents rare high-spending trips. The shape is more extreme than the basket size histogram which means spending inequality between customers is even more pronounced than item count differences.

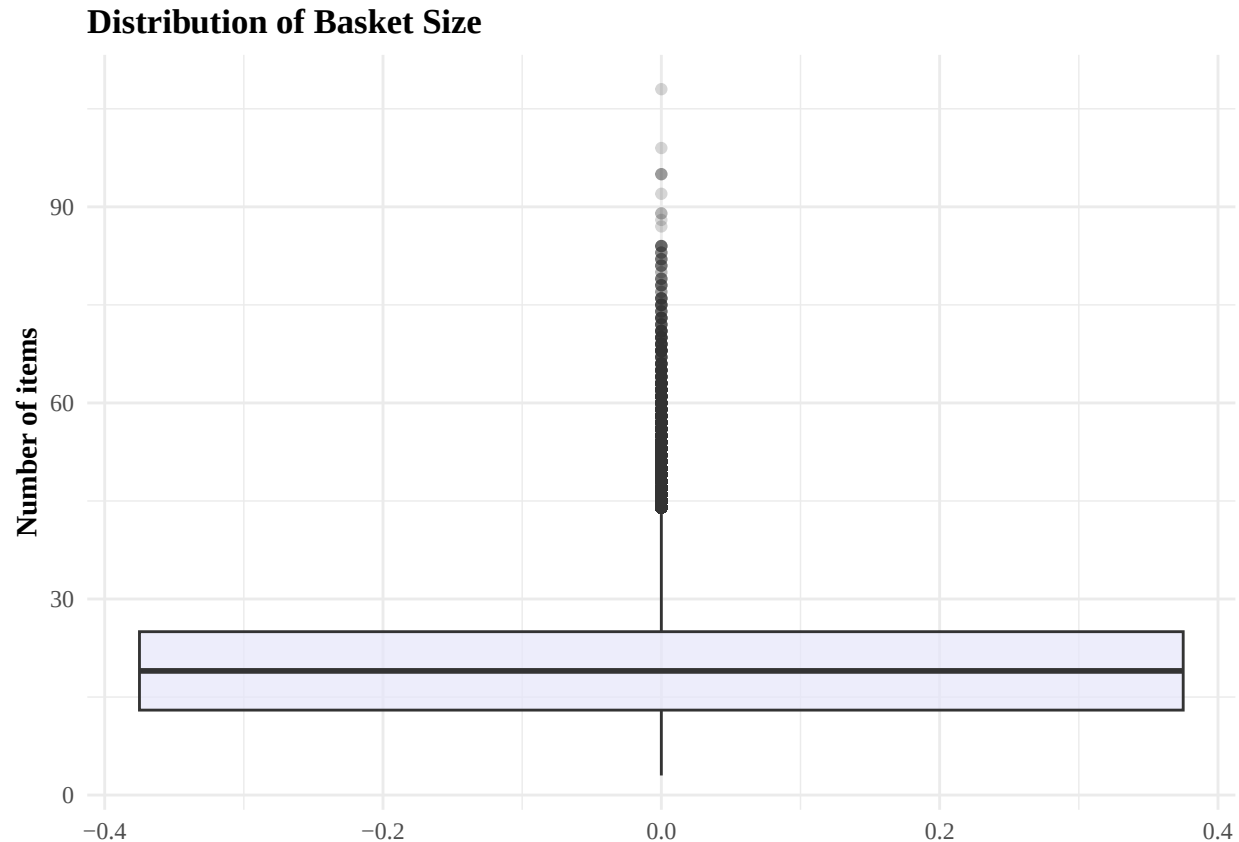
Boxplots of Basket Size and Total Basket Value

```

ggplot(basket_data, aes(y = total_items)) +
  geom_boxplot(fill = "lavender", alpha = 0.7, outlier.alpha = 0.2) +
  labs(
    title = "Distribution of Basket Size",
    y = "Number of items"
  ) +
  theme_minimal(base_family = "serif") +

```

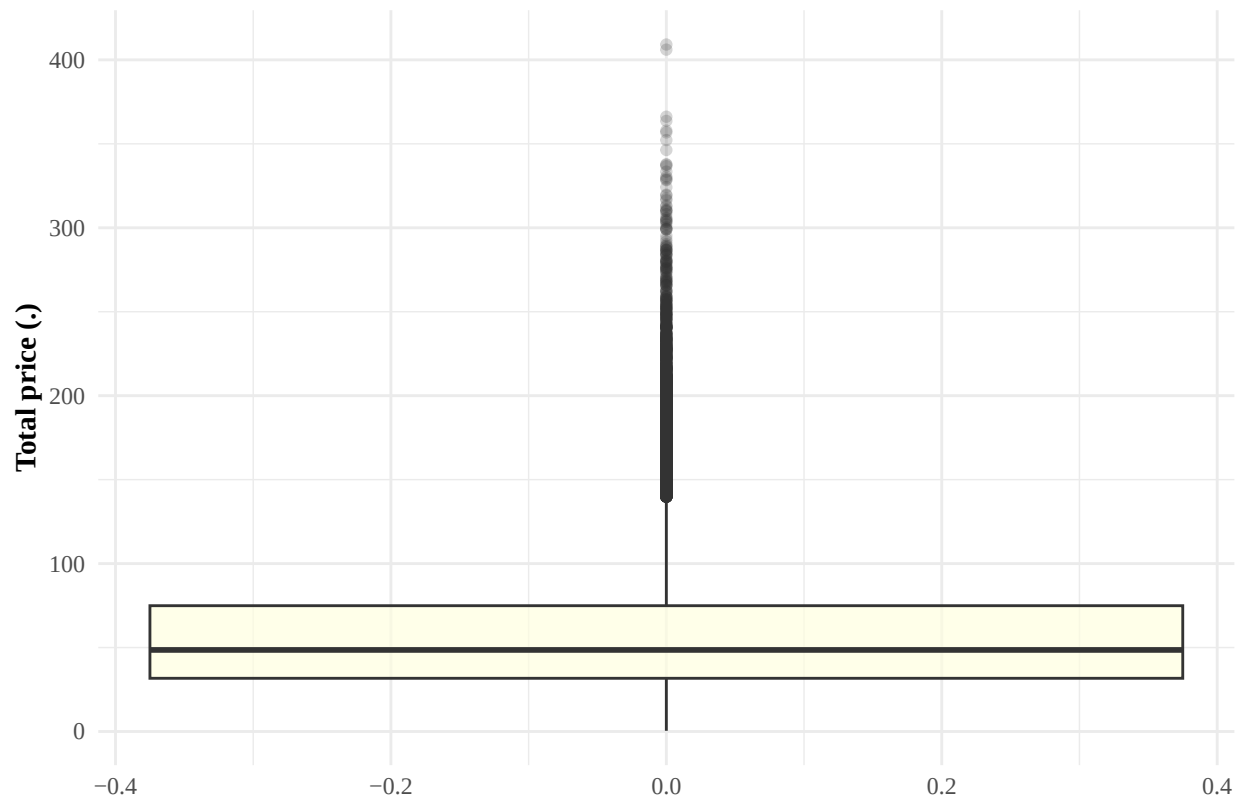
```
theme(
  plot.title = element_text(face = "bold"),
  axis.title = element_text(face = "bold")
)
```



The box (IQR) sits between roughly 13 and 25 items, with the median line at 19. The whiskers extend to about 3–42 items, and everything beyond that is flagged as an outlier. There are many outlier points stretching up to 108, but the box itself is compact, confirming that most customers behave similarly and extreme basket sizes are genuinely unusual, not just rare.

```
ggplot(basket_data, aes(y = total_price)) +
  geom_boxplot(fill = "lightyellow", alpha = 0.7, outlier.alpha = 0.2) +
  labs(
    title = "Distribution of Total Basket Value",
    y = "Total price (€)"
  ) +
  theme_minimal(base_family = "serif") +
  theme(
    plot.title = element_text(face = "bold"),
    axis.title = element_text(face = "bold")
  )
```

Distribution of Total Basket Value



The box spans roughly €32 to €75, with the median around €49. The upper whisker reaches approx. €140, and beyond that there is a very dense cluster of outliers all the way to €409. Notably, the outlier column is much denser here than in the basket size boxplot which means high spending is more common as an anomaly than having a very large basket. Some customers spend disproportionately even without buying unusually many items, likely due to expensive products.

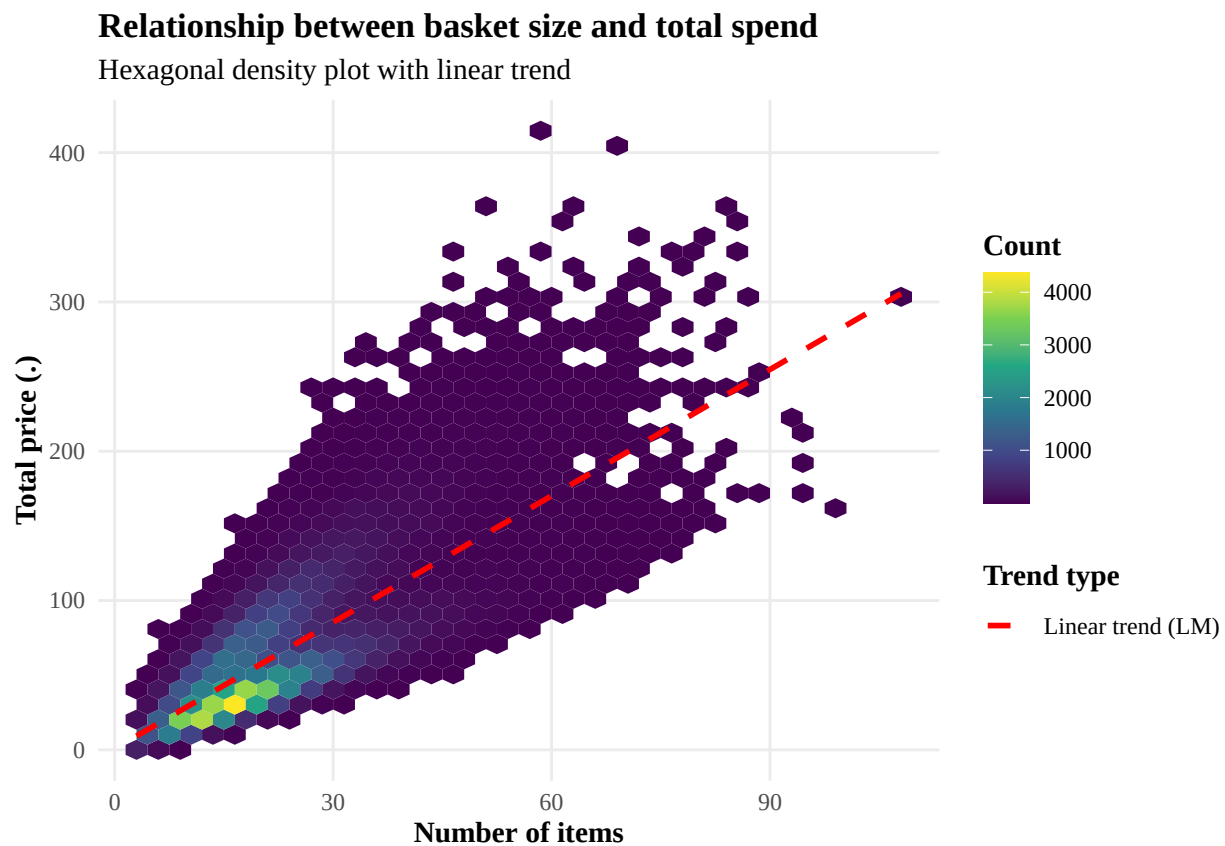
Relationship Between Basket Size and Total Spend

```
ggplot(basket_data, aes(x = total_items, y = total_price)) +  
  geom_hex(bins = 35) +  
  geom_smooth(aes(color = "Linear trend (LM)",  
                  method = "lm",  
                  linetype = "dashed",  
                  se = FALSE) +  
  scale_color_manual(  
    values = c("Linear trend (LM)" = "red"),  
    name = "Trend type"  
  ) +  
  scale_fill_viridis_c(name = "Count") +  
  labs(  
    title = "Relationship between basket size and total spend",  
    subtitle = "Hexagonal density plot with linear trend",  
    x = "Number of items",  
    y = "Total price (€)"
```

```

) +
theme_minimal(base_family = "serif") +
theme(
  plot.title = element_text(face = "bold"),
  axis.title = element_text(face = "bold"),
  legend.title = element_text(face = "bold"),
  panel.grid.minor = element_blank()
)

```



There is a clear positive linear relationship: as the number of items increases, total spend increases. The trend line confirms this. However, the spread of points fans out as basket size grows, meaning large baskets are less predictable in value. Some customers with 50+ items spend relatively little (cheap items), while others spend a lot. For small baskets (under 20 items), spending is tight and predictable.

Time Behavior

Total Scan Time

Distributions To analyze time behavior at the basket level, extreme values in total scan time are removed by excluding the top 1% of transactions. The filtered data is then used to plot the distribution of total scan time using a histogram, a density curve, and a median reference line. A second histogram on a log scale is included to better show the skewness and overall spread of the data.

```

basket_time_filtered <- basket_data %>%
  filter(total_time < quantile(total_time, 0.99, na.rm = TRUE))

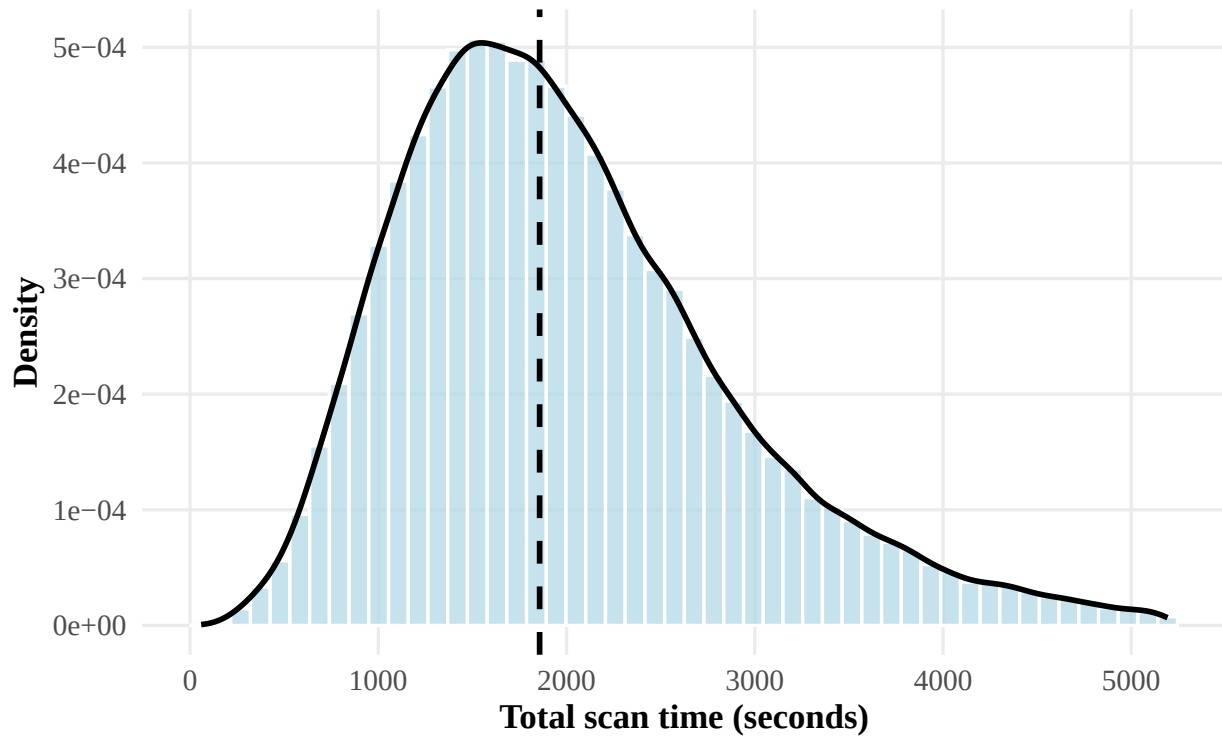
median_time <- median(basket_time_filtered$total_time, na.rm = TRUE)

ggplot(basket_time_filtered, aes(x = total_time)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 50,
    fill = "lightblue",
    color = "white",
    alpha = 0.7
  ) +
  geom_density(color = "black", linewidth = 1) +
  geom_vline(
    xintercept = median_time,
    linetype = "dashed",
    linewidth = 1
  ) +
  labs(
    title = "Distribution of total scan time",
    subtitle = "Top 1% of extreme transactions removed; dashed line indicates median",
    x = "Total scan time (seconds)",
    y = "Density"
  ) +
  theme_minimal(base_size = 13, base_family = "serif") +
  theme(
    plot.title = element_text(face = "bold", size = 16),
    axis.title = element_text(face = "bold"),
    axis.text = element_text(size = 11),
    panel.grid.minor = element_blank()
  )

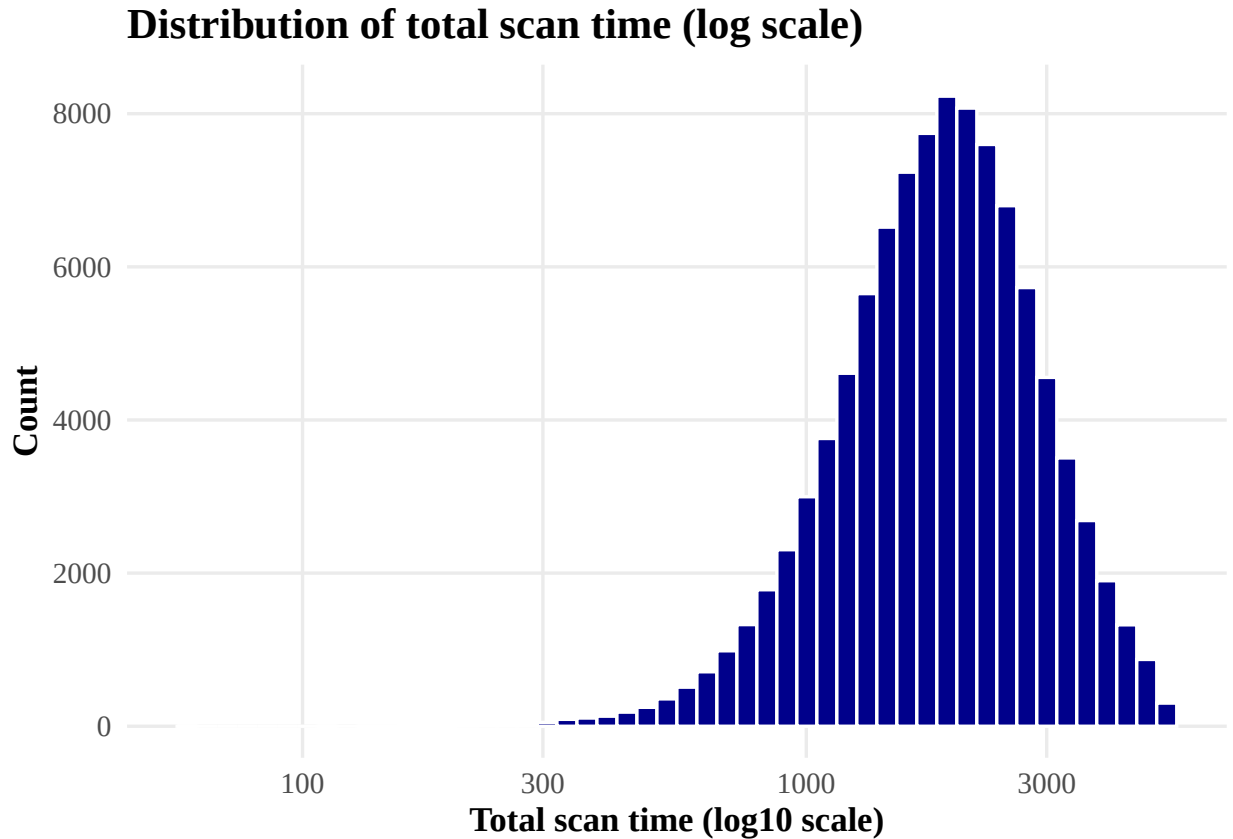
```

Distribution of total scan time

Top 1% of extreme transactions removed; dashed line indicates median



```
ggplot(basket_time_filtered, aes(x = total_time)) +  
  geom_histogram(bins = 50, fill = "darkblue", color = "white") +  
  scale_x_log10() +  
  labs(  
    title = "Distribution of total scan time (log scale)",  
    x = "Total scan time (log10 scale)",  
    y = "Count"  
  ) +  
  theme_minimal(base_size = 13, base_family = "serif") +  
  theme(  
    plot.title = element_text(face = "bold", size = 16),  
    axis.title = element_text(face = "bold"),  
    axis.text = element_text(size = 11),  
    panel.grid.minor = element_blank()  
  )
```



The distribution of total scan time is unimodal and right-skewed, with a clear peak around 1,500–2,000 seconds. The median transaction duration is about 30 minutes, which reflects a typical shopping session.

Even after removing the top 1% of extreme values, the distribution still has a long right tail, showing that shopping time varies quite a lot. Most customers finish within roughly 17 to 40 minutes, but some take much longer, likely due to pauses or different shopping habits.

When using a log scale, the distribution becomes more symmetric. This suggests that total scan time follows a log-normal pattern, meaning it is influenced by several factors at once, such as basket size, scanning speed, and pauses between items. The log transformation also makes the central pattern easier to see by reducing the impact of extreme values.

Overall, this indicates that using log-transformed time variables could be more useful for further analysis and modeling.

Quantiles Key percentiles of total scan time (25th, 50th, 75th, and 95th) are calculated to better understand how transaction durations are distributed. These values help show the typical range of shopping time and highlight longer transactions in the upper part of the distribution.

```
time_quantiles <- quantile(  
  basket_time_filtered$total_time,  
  probs = c(0.25, 0.5, 0.75, 0.95),  
  na.rm = TRUE  
)  
  
time_quantiles
```



```
## 25% 50% 75% 95%
## 1356 1857 2484 3694
```

The distribution of total scan time shows that half of all transactions are completed in about 31 minutes, and 75% finish within 41 minutes.

Most shopping trips fall in the 23 to 41 minute range, which reflects typical behavior. However, the 95th percentile exceeds 60 minutes, showing that there is still quite a bit of variation.

This means that while most customers shop in a fairly consistent way, a smaller group takes much longer, likely due to pauses or more complex shopping trips.

Scanning Pace (average time)

Distributions To analyze scanning pace, extreme values in average time between scans are removed by excluding the top 1% of observations. The filtered data is then used to plot the distribution with a histogram, a density curve, and a median reference line. This helps show the typical scanning behavior as well as how much it varies across transactions.

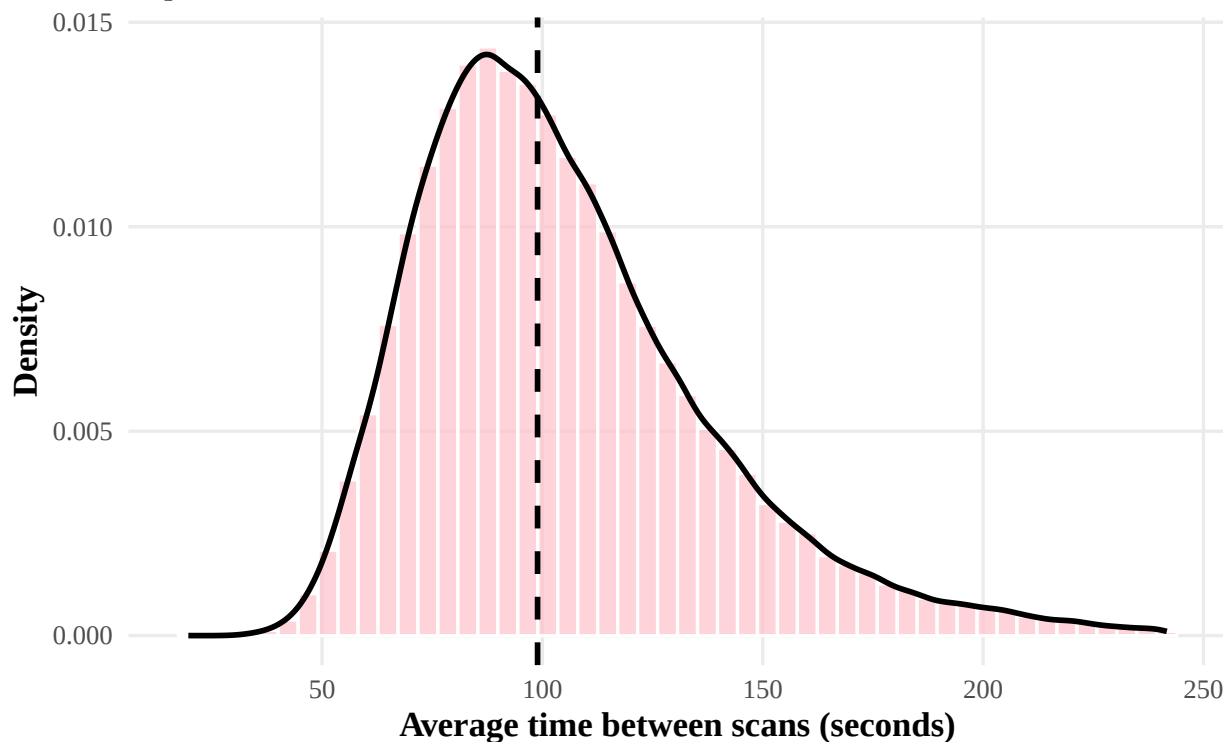
```
basket_speed_filtered <- basket_data %>%
  filter(avg_time_per_item < quantile(avg_time_per_item, 0.99, na.rm = TRUE))

median_speed <- median(basket_speed_filtered$avg_time_per_item, na.rm = TRUE)

ggplot(basket_speed_filtered, aes(x = avg_time_per_item)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 50,
    fill = "pink",
    color = "white",
    alpha = 0.7
  ) +
  geom_density(color = "black", linewidth = 1) +
  geom_vline(
    xintercept = median_speed,
    linetype = "dashed",
    linewidth = 1
  ) +
  labs(
    title = "Distribution of average time between scans",
    subtitle = "Top 1% removed; dashed line indicates median",
    x = "Average time between scans (seconds)",
    y = "Density"
  ) +
  theme_minimal(base_size = 13, base_family = "serif") +
  theme(
    plot.title = element_text(face = "bold"),
    axis.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )
```

Distribution of average time between scans

Top 1% removed; dashed line indicates median



The distribution of average time between scans is unimodal and right-skewed, with a clear peak around 80-100 seconds. The median of about 95-100 seconds suggests that customers do not scan items continuously, but take pauses between scans as part of the shopping process.

Most transactions fall within the range of 70 to 130 seconds per item, which reflects typical behavior. However, the long right tail indicates that some customers take much longer between scans, likely due to interruptions or slower shopping patterns.

Overall, this shows that average time per item is not just a measure of scanning speed, but a behavioral indicator of how customers move through the store.

Quantiles Key percentiles of the average time between scans (25th, 50th, 75th, and 95th) are calculated to better understand how scanning pace is distributed. These values help show the typical range of behavior and highlight slower scanning patterns in the upper part of the distribution.

```
speed_quantiles <- quantile(  
  basket_speed_filtered$avg_time_per_item,  
  probs = c(0.25, 0.5, 0.75, 0.95),  
  na.rm = TRUE  
)
```

```
speed_quantiles
```

```
##      25%      50%      75%      95%  
## 80.94444 98.90909 122.11275 168.70600
```

The distribution of average time between scans shows that half of all transactions are completed in about 99 seconds per item, with most falling between 81 and 122 seconds. This relatively narrow range suggests that scanning behavior is quite consistent for most customers.

However, the upper tail reaches around 169 seconds at the 95th percentile, indicating that some customers take much longer between scans, likely due to pauses or interruptions.

These patterns make it possible to distinguish between fast, typical, and slower scanning customers.

Average time between scans vs Basket size

LOESS

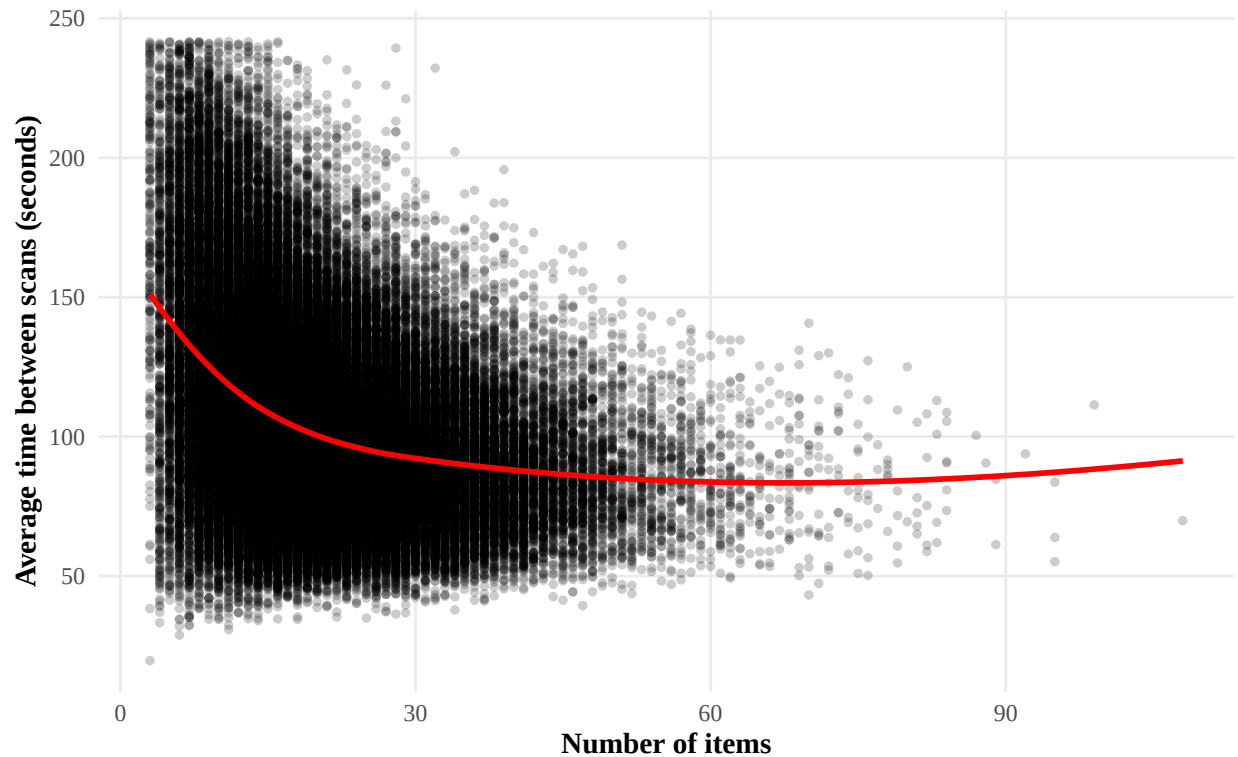
The relationship between basket size and scanning pace is explored using scatter plots with a LOESS smoothing line. The full dataset gives an overall view, while a random sample of 5,000 transactions makes the plot easier to read.

The trend line shows how the average time between scans changes with the number of items, helping to reveal any non-linear patterns in customer behavior.

```
#loess
ggplot(basket_speed_filtered, aes(x = total_items, y = avg_time_per_item)) +
  geom_point(alpha = 0.2, size = 1) +
  geom_smooth(method = "loess", se = FALSE, color = "red", linewidth = 1) +
  labs(
    title = "Average time between scans vs Basket size",
    subtitle = "Trend line shows how average scan intervals vary with basket size",
    x = "Number of items",
    y = "Average time between scans (seconds)"
  ) +
  theme_minimal(base_family = "serif") +
  theme(
    plot.title = element_text(face = "bold"),
    axis.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )
```

Average time between scans vs Basket size

Trend line shows how average scan intervals vary with basket size



```
#sample 5,000

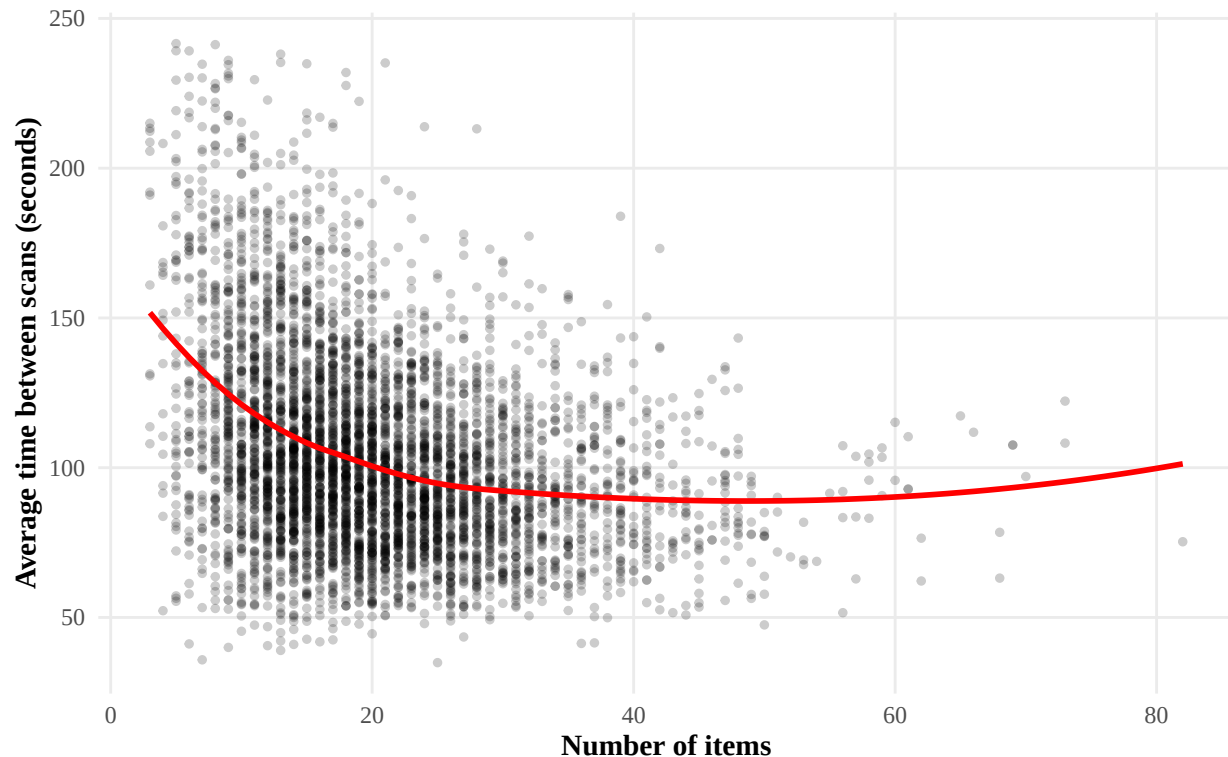
set.seed(123)

basket_sample <- basket_speed_filtered %>%
  slice_sample(n = 5000)

ggplot(basket_sample, aes(x = total_items, y = avg_time_per_item)) +
  geom_point(alpha = 0.2, size = 1) +
  geom_smooth(method = "loess", se = FALSE, color = "red", linewidth = 1) +
  labs(
    title = "Average time between scans vs Basket size",
    subtitle = "Based on a random sample of 5,000 transactions",
    x = "Number of items",
    y = "Average time between scans (seconds)"
  ) +
  theme_minimal(base_family = "serif") +
  theme(
    plot.title = element_text(face = "bold"),
    axis.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )
```

Average time between scans vs Basket size

Based on a random sample of 5,000 transactions



The relationship between basket size and average time between scans is not linear.

For small baskets, customers are slower, often taking more than 120 seconds per item, likely because they are browsing or deciding what to buy.

As basket size increases, the average time per item decreases to around 85-100 seconds, suggesting that customers get into a more consistent shopping rhythm.

For very large baskets, the pattern levels off and slightly increases, which could be due to fatigue or more complex shopping.

Overall, this shows that scanning speed is not constant, but depends on basket size.

Scanning Behavior

Scanning behavior is further explored by comparing small and large baskets, showing clear differences in how the average time between scans is distributed.

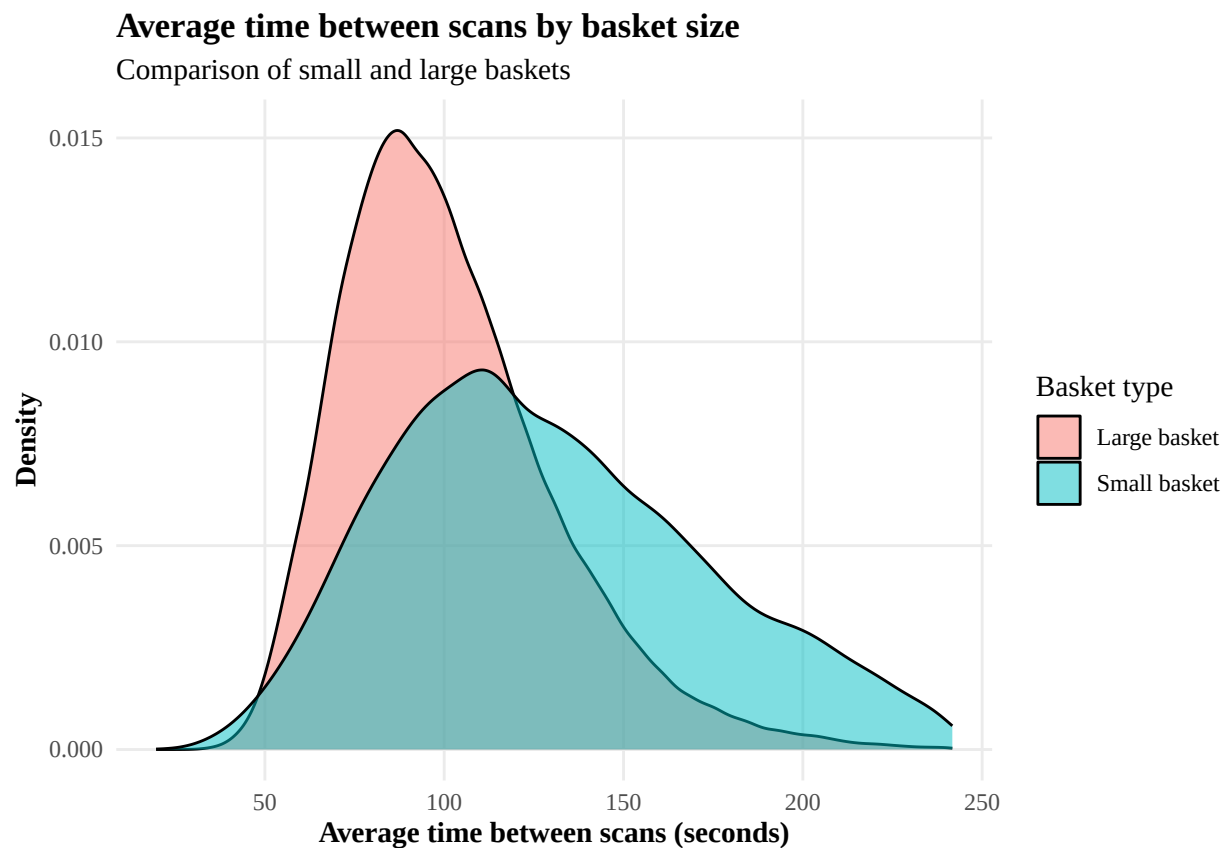
In addition, the relationship between basket size and scanning pace is examined using the median and interquartile range, which helps show how both the typical value and the variability of scan intervals change as the number of items increases.

```
basket_speed_filtered %>%
  mutate(size_group = ifelse(total_items <= 10, "Small basket", "Large basket")) %>%
  ggplot(aes(x = avg_time_per_item, fill = size_group)) +
  geom_density(alpha = 0.5) +
  labs(
    title = "Average time between scans by basket size",
```

```

    subtitle = "Comparison of small and large baskets",
    x = "Average time between scans (seconds)",
    y = "Density",
    fill = "Basket type"
  ) +
  theme_minimal(base_family = "serif") +
  theme(
    plot.title = element_text(face = "bold"),
    axis.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )

```

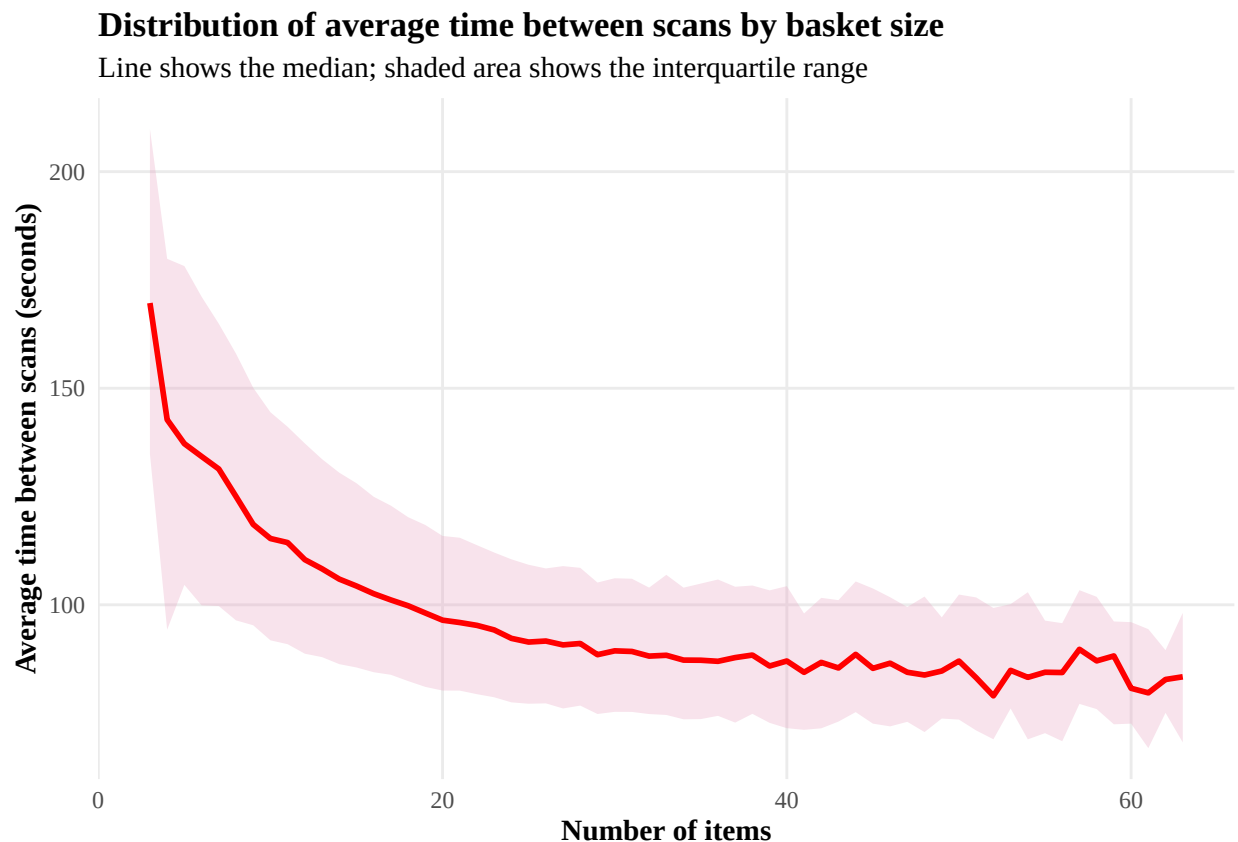


```

basket_speed_filtered %>%
  group_by(total_items) %>%
  summarise(
    n = n(),
    q25 = quantile(avg_time_per_item, 0.25, na.rm = TRUE),
    q50 = median(avg_time_per_item, na.rm = TRUE),
    q75 = quantile(avg_time_per_item, 0.75, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  filter(n >= 30) %>%
  ggplot(aes(x = total_items)) +
  geom_ribbon(aes(ymin = q25, ymax = q75), alpha = 0.3, fill = "#E8A0BF") +
  geom_line(aes(y = q50), color = "red", linewidth = 1) +

```

```
labs(
  title = "Distribution of average time between scans by basket size",
  subtitle = "Line shows the median; shaded area shows the interquartile range",
  x = "Number of items",
  y = "Average time between scans (seconds)"
) +
theme_minimal(base_family = "serif") +
theme(
  plot.title = element_text(face = "bold"),
  axis.title = element_text(face = "bold"),
  panel.grid.minor = element_blank()
)
```



The comparison between small and large baskets shows clear differences in scanning behavior.

Small baskets have a wider spread and higher variability in the time between scans, suggesting more exploratory and less structured shopping. In contrast, large baskets are more concentrated around 80–100 seconds, indicating more consistent and efficient behavior.

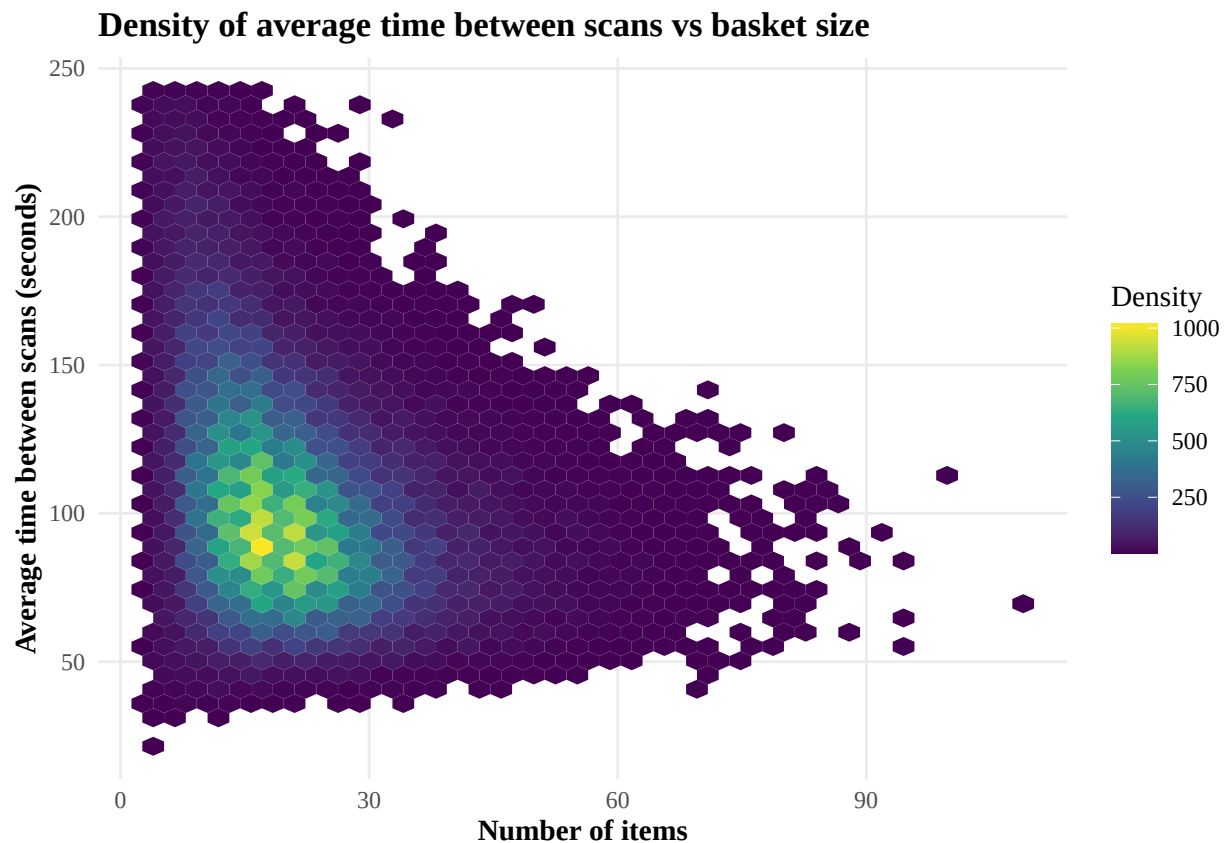
Looking at basket size more closely, the average time per item drops quickly for small baskets, then stabilizes for medium-sized ones, and slightly increases again for very large baskets. At the same time, variability decreases as basket size grows, meaning behavior becomes more predictable in larger transactions.

Density

A hexbin plot is used to show how observations are distributed across basket size and average time between scans. Instead of plotting individual points, the data is grouped into hexagonal bins, making it easier to see

where most transactions are concentrated and where less common patterns occur.

```
ggplot(basket_speed_filtered, aes(x = total_items, y = avg_time_per_item)) +  
  geom_hex(bins = 40) +  
  scale_fill_viridis_c() +  
  labs(  
    title = "Density of average time between scans vs basket size",  
    x = "Number of items",  
    y = "Average time between scans (seconds)",  
    fill = "Density"  
  ) +  
  theme_minimal(base_family = "serif") +  
  theme(  
    plot.title = element_text(face = "bold"),  
    axis.title = element_text(face = "bold"),  
    panel.grid.minor = element_blank()  
  )
```



The density plot shows a clear concentration of observations for baskets with 10 to 30 items, where average scan times are around 80 to 110 seconds per item. This represents typical customer behavior.

There is a negative relationship between basket size and average time per item, meaning that customers become more efficient as the number of items increases. At the same time, variability decreases with basket size, suggesting that larger baskets are associated with more consistent and structured shopping.

Very slow scan times are mostly found in small baskets, likely reflecting browsing rather than inefficiency.

To sum up, these results show that scanning behavior depends on basket size and should be taken into account when analyzing behavior and detecting unusual patterns.